Well-Typed

Compiling C to Haskell

How `hs-bindgen` Translates C Headers

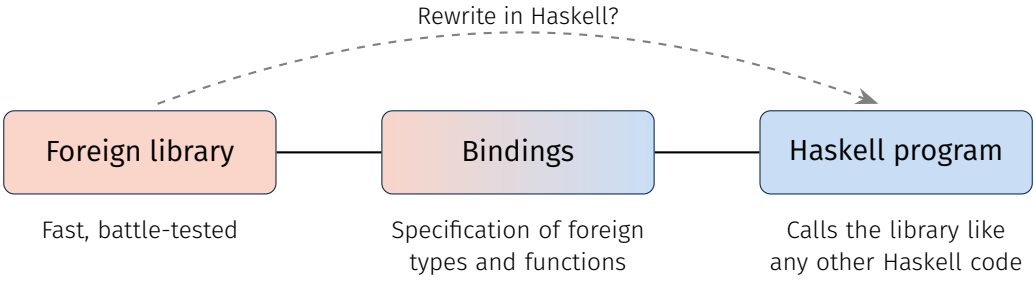
Dominik Schrempf

Well-Typed

MuniHac, October 11, 2026

Using a library from another language

Well-Typed



⚠️ Writing bindings by hand is tedious and error-prone: mismatches may surface at run time, and the bindings must keep up with upstream changes.

In practice, the common ground is C: the Haskell foreign function interface (FFI) understands C, and other languages export C interfaces.

What is `hs-bindgen`?

Well-Typed

`hs-bindgen` …

- automatically generates Haskell bindings from C header files.
- leverages the power of `libclang` (parsing, layout, offsets, …).
- is structured like a compiler.
- extracts and translates documentation.
- supports translation of multiple C libraries in a compositional way.
- even typechecks C macros (*best effort*).
- is a team effort, with many community contributions.
- is on Hackage: version 1.0.0.0 was released a few days ago 🎉.

Real-world example: `libpcap`

Well-Typed

Excerpt from `pcap.h`

```
typedef struct pcap_if pcap_if_t;

PCAP_API int    pcap_findalldevs(pcap_if_t **, char *);

// Item in a list of interfaces.
struct pcap_if {
  struct pcap_if   *next;
  char             *name;        /* name to hand to "pcap_open_live()" */
  char             *description; /* textual description of interface, or NULL */
  struct pcap_addr *addresses;
  bpf_u_int32       flags;       /* PCAP_IF_ interface flags */
};

// Representation of an interface address.
struct pcap_addr {
  struct pcap_addr *next;
  struct sockaddr  *addr;        /* address */
  struct sockaddr  *netmask;     /* netmask for that address */
  struct sockaddr  *broadaddr;   /* broadcast address for that address */
  struct sockaddr  *dstaddr;     /* P2P destination address for that address */
};
```

Real-world example: `libpcap`

Well-Typed

Generate bindings with `hs-bindgen`

```
flags=(
    -I "${PCAP_INCLUDE_DIR}"
    --module Generated.Pcap
    --omit-field-prefixes
    --clang-option '-std=gnu17'
    --select-by-header-path pcap.h
    --enable-program-slicing
    --select-except-deprecated
    # ... (Project files bundling these flags are in the works.)
)
hs-bindgen-cli preprocess "${flags[@]}" pcap.h
```

The resulting Haskell modules expose, for example,

```
{-| __C declaration:__ @pcap_findalldevs@
    __defined    at:__ @pcap\/pcap.h 795:14@
    __exported   by:__ @pcap.h@
-}
pcap_findalldevs :: Ptr (Ptr Pcap_if_t) -> Ptr CChar -> IO CInt
pcap_findalldevs = _
```

Real-world example: `libpcap`

Well-Typed

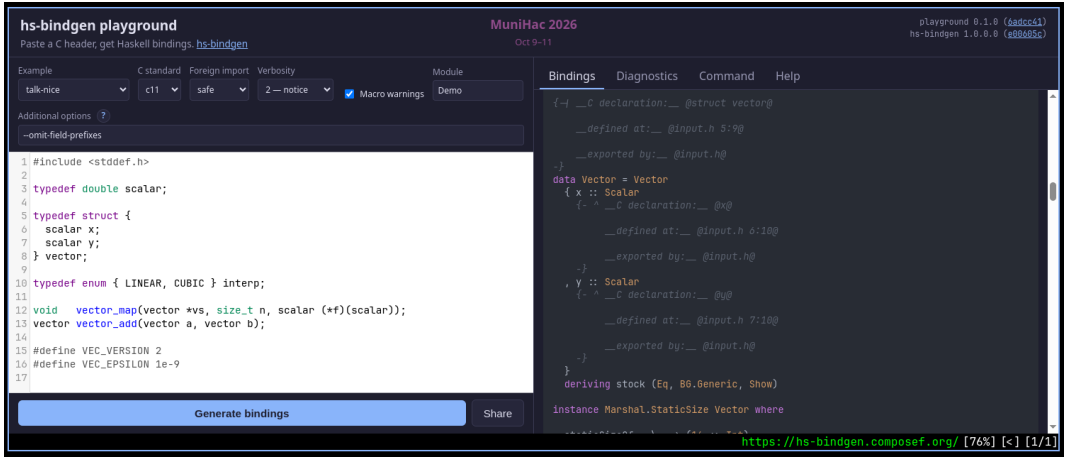
Capture live packets using Haskell

```
Running the project with elevated privileges (packet capture)

List of network devices found:
  - wlp0s20f3
  [...]

Capturing packets on wlp0s20f3 (press Ctrl-C to stop)...

#     Timestamp                Bytes  Description
#1    1785143786.567790     86 bytes  ac:22:05:dc:6c:3c -> 64:49:7d:84:39:38  IPv6
#2    1785143786.858210    182 bytes  ac:22:05:dc:6c:3c -> 33:33:00:00:00:01  IPv6
#3    1785143787.290057   2032 bytes  64:49:7d:84:39:38 -> ac:22:05:dc:6c:3c  IPv6
#4    1785143787.290901    950 bytes  64:49:7d:84:39:38 -> ac:22:05:dc:6c:3c  IPv6
#5    1785143787.291761    116 bytes  64:49:7d:84:39:38 -> ac:22:05:dc:6c:3c  IPv6
[...]
```



The `hs-bindgen` playground, accessible at hs-bindgen.composef.org.

Classic example C header

Well-Typed

```c
#include <stddef.h>

typedef double scalar;

typedef struct {
  scalar x;
  scalar y;
} vector;

typedef enum { LINEAR, CUBIC } interp;

void   vector_map(vector *vs, size_t n, scalar (*f)(scalar));
vector vector_add(vector a, vector b);

#define VEC_VERSION 2
#define VEC_EPSILON 1e-9
```

Type definitions

Well-Typed

```c
typedef double scalar;
```

```haskell
newtype Scalar = Scalar { unwrap :: CDouble }
  deriving stock   (Eq, Generic, Ord, Read, Show)
  deriving newtype (Enum, Floating, Fractional, HasFFIType, Num, Prim,
                    ReadRaw, Real, RealFloat, RealFrac, StaticSize,
                    Storable, WriteRaw)
```

Type definitions become `newtype`s with rich sets of instances.

Structures (`struct`)

Well-Typed

Automated layout retrieval

```
typedef struct { scalar x; scalar y; } vector;
```

```
-- Omitting field prefixes is a choice.
data Vector = Vector { x :: Scalar, y :: Scalar }
  deriving stock (Eq, Generic, Show)

instance StaticSize Vector where
  staticSizeOf    = \_ -> (16 :: Int)
  staticAlignment = \_ -> (8 :: Int)

instance HasCField Vector "y" where
  offset# = \_ -> \_ -> 8  -- bytes
```

- Sizes, alignments, and offsets come from **`libclang`**. This is a main reason why **`hs-bindgen`** is a **`libclang`** consumer and not a C header parser.
- Unlike **`scalar`**, a **`typedef`** is squashed into its **`struct`**, **`enum`**, or **`union`** if both share a name or the **`typedef`** is its only use: one type **`Vector`**.

Pointer manipulation

Well-Typed

Change one field without marshalling the record

```haskell
data Vector = Vector { x :: Scalar, y :: Scalar }
  deriving stock (Eq, Generic, Show)
instance HasCField Vector "y" where
  offset# = \_ -> \_ -> 8  -- bytes
```

`hs-bindgen` generates `HasCField` instances for `Vector`, and uses them to generate `HasField` instances for `Ptr Vector`:

```haskell
instance HasField "y" (Ptr Vector) (Ptr Scalar) where
  getField = HasCField.fromPtr (Proxy @"y")  -- pointer + offset
```

With record dot syntax:

User code

```haskell
setY :: Ptr Vector -> IO ()
setY p = poke p.y 3
```

Only `y` is written; the `Vector` is never marshalled to a Haskell value and back.

Enumerations (`enum`)

Well-Typed

The obvious Haskell translation is wrong

```
typedef enum { LINEAR, CUBIC } interp;
```

```
newtype Interp = Interp { unwrap :: CUInt }

pattern LINEAR :: Interp; pattern LINEAR = Interp 0
pattern CUBIC  :: Interp; pattern CUBIC  = Interp 1

instance CEnum           Interp where declaredValues = _
instance SequentialCEnum Interp where -- ...
instance Show            Interp where -- declared via 'CEnum'
```

- A C `enum` declares *values*; it does not constrain the *type*. `interp i = 47;` is legal C. Hence, no ADT, no derived `Enum` or `Bounded`, and the pattern synonyms are deliberately not `COMPLETE`.
- The class `CEnum` records the declared values: `show CUBIC` is `"CUBIC"`, `show (Interp 47)` is `"Interp 47"`.
- If the `enum` is in fact `closed` (e.g., a Rust `enum` exposed through C), the user can say so; then the patterns are declared `COMPLETE`.

Function pointers and `size_t`

Well-Typed

```c
void   vector_map(vector *vs, size_t n, scalar (*f)(scalar));
```

```haskell
foreign import ccall safe "vector_map"
  vector_map_base :: Ptr Void -> CSize -> FunPtr Void -> IO ()

vector_map :: Ptr Vector -> CSize -> FunPtr (Scalar -> IO Scalar) -> IO ()
vector_map = _ -- Implementation converts arguments and calls `vector_map_base`.
```

We do not generate a binding for `size_t`. Instead, *external binding specifications* tell `hs-bindgen` that a binding already exists: `size_t` becomes `CSize`.

`scalar (*f)(scalar)` becomes a `FunPtr`. `hs-bindgen` generates a `ToFunPtr` instance, so `withFunPtr` passes a Haskell function:

User code

```haskell
doubleVec :: Ptr Vector -> CSize -> IO ()
doubleVec vs = withFunPtr (pure . (* 2)) . vector_map vs
```

Safe and unsafe

Well-Typed

`unsafe` Faster; must not call back into Haskell; no garbage collection during the call.

`safe` Slower because of RTS bookkeeping; can call back into Haskell; potential garbage collection during the call.

`hs-bindgen` cannot know which one is appropriate, and generates both, in separate modules (e.g., `Generated.Pcap.Safe` and `Generated.Pcap.Unsafe`).

For example, `vector_map` must be called `safe` if its `FunPtr` wraps a Haskell function: C then calls back into Haskell.

Passing `structs` by value

Well-Typed

How do you write a **`foreign import`** for this function?

```
1 vector vector_add(vector a, vector b);
```

You cannot. The Haskell FFI passes only integers, floats, and pointers. How a **`struct`** is handled (e.g., whether it is stored in a register, or the stack) depends on its fields and on the platform, and GHC does not know these rules.

Write a C wrapper yourself.

Passing `structs` by value

Well-Typed

```c
vector vector_add(vector a, vector b);

// Generated C wrapper: structs passed by pointer.
void vector_add_fi(vector *arg1, vector *arg2, vector *arg3) {
  *arg3 = (vector_add)(*arg1, *arg2);
}
```

```haskell
foreign import ccall safe "vector_add_fi"
  vector_add_base :: Ptr Void -> Ptr Void -> Ptr Void -> IO ()

vector_add :: Vector -> Vector -> IO Vector
vector_add v0 v1 = _ -- Implementation converts arguments to pointers and
                     -- calls `vector_add_base`.
```

Template Haskell (`addCSource`) emits the C wrapper.

Macros

Well-Typed

```
#define VEC_VERSION 2
#define VEC_EPSILON 1e-9
```

```
vEC_VERSION :: CInt
vEC_VERSION = (2 :: CInt)

vEC_EPSILON :: CDouble
vEC_EPSILON = (1.0e-9 :: CDouble)
```

A *typechecker* infers and checks the types, imposing C's rules for expressions on macros.

Actual classic example C header

Well-Typed

```c
#include <stddef.h>

#define MASK   (1u << 31)     // ?
#define ADD(x) ((x) + 4)      // Function-like macro

// Bitfield
struct blinds {
  char window_id;
  int  tilt : 1;              // `tilt` and `close_blinds` are only one bit wide.
  int  close_blinds : 1;
  long timestamp;
};

// Structure with flexible array member (FLAM)
struct surname {
  size_t len;
  char   data[];              // `hs-bindgen` cannot know the size of the `struct`
};                            // because the FLAM has an unspecified number of
                              // elements.
```

Macros, take two

Well-Typed

```c
#define MASK   (1u << 31)
#define ADD(x) ((x) + 4)
```

```haskell
mASK :: CUInt
mASK = (C.Expr.HostPlatform.<<) (1 :: CUInt) (31 :: CInt)

aDD :: forall a0. Add a0 CInt ⇒ a0 -> AddRes a0 CInt
aDD = \x0 -> (C.Expr.HostPlatform.+) x0 (4 :: CInt)
```

- A pluggable *macro language*, by default `c-expr`, parses and typechecks the macro body, following C's integral promotions and arithmetic conversions: `MASK` is `CUInt`.
- A function-like macro becomes a *class-polymorphic* function: `ADD` adds to an integer or shifts a pointer.

Bitfields

Well-Typed

```
struct blinds { char window_id; int tilt : 1; /* ... */ long timestamp; };
```

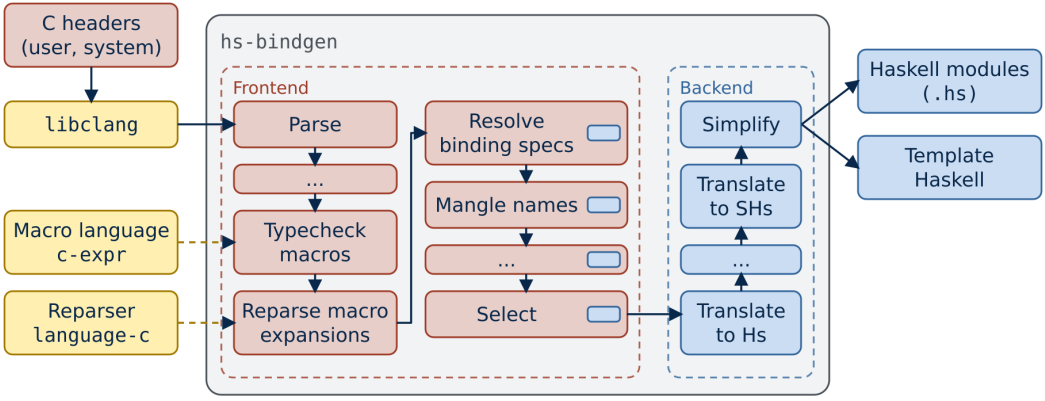
```
data Blinds = Blinds
  { window_id    :: CChar, tilt      :: CInt
  , close_blinds :: CInt,  timestamp :: CLong }

instance HasCBitfield Blinds "tilt" where
  type CBitfieldType Blinds "tilt" = CInt
  bitfieldOffset# = \_ -> \_ -> 8  -- bits, not bytes
  bitfieldWidth#  = \_ -> \_ -> 1  -- bits
```

- Bitfields become normal-looking records; only the *marshalling* code reflects the packed structure.
- `long timestamp` is `CLong`: width depends on the platform *and* the compiler.

hs-bindgen is a compiler

Well-Typed



Some of the passes. Roughly: Red: C world. Blue: Haskell world. Yellow: external tools.

Trees That Grow 13 frontend passes, each indexed by a type: GHC typechecks the pass inputs and outputs.

Two modes *Preprocess* writes `.hs` files; *Template Haskell* uses a splice.

Macros in the pipeline

Well-Typed

Definitions and invocations are different problems

```c
#define VEC_DIM 2
#define REAL double
REAL vec_norm(REAL x, REAL y);
```

```haskell
vEC_DIM :: CInt
newtype REAL = REAL { unwrap :: CDouble }
vec_norm :: REAL -> REAL -> IO REAL
```

1. Typecheck macros: classify the *definitions*; `VEC_DIM` is *value-like*; `REAL` is *type-like*, treated like a `typedef`.
2. Reparse macro expansions: `libclang` sees `double vec_norm(double x, double y)`; the macro is gone. `hs-bindgen` reparses the *declaration* with `REAL` unexpanded, treating it like a `typedef` reference.

Reparsing needs to know *which macros are types*, so the typechecker runs first.

Prescriptive binding specifications

Describe bindings of a Haskell module

Well-Typed

Say we dislike the name `Vector`:

```
ctypes:
- headers: vector.h
  cname: struct vector
  hsname: CVector
```

```
data CVector = CVector { x :: Scalar, y :: Scalar }
```

- Also prescribable: the `representation` (`record`, `newtype`, `emptydata`), which `instances` to derive, and whether an `enum` is `closed` (`COMPLETE` pragma).
- `hs-bindgen` can emit the specification used so the user can edit and feed back.

External binding specifications

Well-Typed

The binding already exists: reuse it, so bindings compose

```c
#include "vector.h"                              // vector_rotate.h
vector *vector_rotate(vector *v, double th);
```

```haskell
module Vector.Rotate (vector_rotate) where
import qualified Vector

vector_rotate :: Ptr Vector.Vector -> CDouble -> IO (Ptr Vector.Vector)
```

- Without the specification, *every* module defines its own `Vector`, so the modules do not work together.
- Handed the specification of `Vector`, `hs-bindgen` knows the Haskell type already exists, and which module to import it from (and which instances it has).
- The standard library external binding specifications map, for example, `size_t` to `CSize`.

Name mangling

C and Haskell names have different constraints

Well-Typed

C	Haskell	Note
`struct pcap_if`, `pcap_if_t`	`Pcap_if_t`	the `typedef` is *squashed*
a field `name`	`pcap_if_t_name`	by default, fields are top-level[1]
`#define VEC_VERSION`	`vEC_VERSION`	terms start lowercase
`int data(int x)`	`data'`	`data` is a keyword

- Predictable: no numbering; names derive from the C names alone. Adding a field does not unnecessarily rename other fields.
- Minimize clashes: for example, use `data'`, which is invalid C.
- Detect clashes across the translation unit: drop declarations (and their dependents) with a warning. Prescriptive binding specifications can rename them.

[1] `--omit-field-prefixes` yields `name` instead (and enables `DuplicateRecordFields` and `NoFieldSelectors`); all examples in this talk use it.

Selection and program slicing

Well-Typed

Selection Generate bindings for all declarations in the header; or select declarations by hand. Drop what is not required (e.g., deprecated declarations).

Program slicing Automatically pull in transitive dependencies.

```
1 struct point { long double x, y; };
2 struct shape { struct point *pts; size_t n; };
3 double area(const struct shape *s);
```

Select `area`; program slicing adds `shape`, then `point`.

`Select` runs last: declarations `hs-bindgen` cannot support are deselected along with everything that depends on them. The generated code always compiles.

Limitations

Well-Typed

Low-level `hs-bindgen` generates code that is *faithful*, not necessarily *ergonomic*: `IO` operations and C types (e.g., `Ptr`, `CInt`).

- A high-level part (future work) must be heuristic: is the target of a `char *` a `ByteString`, a `String`, or a buffer?
- We are working on an intermediate level: combinators that make handwriting high-level bindings easier.

Not portable Offsets, for example, depend on the platform. Hence, `hs-bindgen` is part of the build pipeline.

- Template Haskell mode is part of the build pipeline by construction.
- We are working on proper tooling support for preprocess mode (in the meantime, we have `nix` 😇).

Alternatives

The user writes the bindings; a tool assists

Well-Typed

`c2hs` Starts from a Haskell module with *hooks* `{#…#}`, a small DSL that `c2hs` expands to Haskell. The `{#fun#}` hook writes higher-level wrappers, but only for a fixed set of patterns. Computes layout itself instead of asking the C compiler.

`hsc2hs` Also starts from a Haskell module, which becomes a C program that prints the module back out, with placeholders `#size`, `#peek`, `#poke` filled in by the C compiler. Layout is correct; declarations are handwritten.

`inline-c` C snippets in quasi-quotes; Template Haskell emits the `foreign import` and a `.c` file.

Alternatives

A tool writes the bindings

Well-Typed

`c2hsc` Reads C headers and writes `.hsc` files, so the bindings are portable. Feature-rich: `hs-bindgen` came a long way before it could translate the complete `c2hsc` test suite.

Generators `haskell-gi`, `vulkan`, `gl`, `amazonka` generate from domain-specific, *rich API descriptions* (e.g., XML specifications), not from headers. Plan: such descriptions may feed the heuristics of a high-level interface.

`hs-bindgen` reads C headers and writes Haskell modules, similar to `rust-bindgen`, its main inspiration.

Thank you!

Well-Typed

`hs-bindgen` team

- Travis Cardwell
- Joris Dral
- Matthias Heinzl
- Armando Santos
- Dominik Schrempf
- Edsko de Vries
- Alex Washburn

Read more

Automatic C Bindings Generation for Haskell, Haskell Symposium 2025.

Past members

- Sam Derbyshire
- Oleg Grenrus

Try `hs-bindgen`!

- `well-typed/hs-bindgen`
- `well-typed/hs-bindgen-tutorial-nix`
- `dschrempf/hs-bindgen-playground`

Structures with flexible array members

Well-Typed

```c
struct surname { size_t len; char data[]; };
```

```haskell
data Surname_Aux = Surname { len :: CSize }

instance FLAM.Offset CChar Surname_Aux where
  offset = \_proxy0 -> 8  -- bytes

type Surname = FLAM.WithFlam CChar Surname_Aux

-- (Available via `hs-bindgen-runtime`.)
data WithFlam elem aux = WithFlam { aux :: aux, flam :: Vector elem }
```

`hs-bindgen` knows *where* the array starts.

To get the marshalling code (`peek`/`poke`), the user has to provide the number of elements:

User code

```haskell
instance FLAM.NumElems CChar Surname_Aux where
  numElems s = fromIntegral s.len
```

Revisiting the `libpcap` example

Well-Typed

A handwritten high-level binding

User code

```
import Generated.Pcap      qualified as Pcap
import Generated.Pcap.Safe qualified as Pcap

findAllDevNames :: IO [String]
findAllDevNames = bracket findAll Pcap.pcap_freealldevs (go [])
  where
    findAll :: IO (Ptr Pcap.Pcap_if_t)
    findAll = alloca $ \pcapIfTPtrPtr ->
      allocaBytes (fromIntegral Pcap.pCAP_ERRBUF_SIZE) $ \errBuf -> do
        success <- Pcap.pcap_findalldevs pcapIfTPtrPtr errBuf
        unless (success == 0) $ fail =<< C.peekCString errBuf
        peek pcapIfTPtrPtr

    go :: [String] -> Ptr Pcap.Pcap_if_t -> IO [String]
    go acc ptr
      | ptr == nullPtr = pure (reverse acc)
      | otherwise = do
          devName <- C.peekCString =<< peek ptr.name
          go (devName : acc) =<< peek ptr.next
```